



11

VOLUME RENDERING



Magnetic resonance imaging (MRI) and computed tomography (CT) scans are diagnostic processes that create *volumetric data*, data that consists of a set of 2D images showing cross sections through a 3D volume. *Volume rendering* is a computer graphics technique used to construct 3D images from this type of volumetric data. Although volume rendering is commonly used to analyze medical scans, it can also be used to create 3D scientific visualizations in academic disciplines such as geology, archaeology, and molecular biology.

The data captured by MRI and CT scans typically follows the form of a 3D grid of dimensions $N_x \times N_y \times N_z$. In other words, there are N_z 2D “slices,” where each slice is an image of size $N_x \times N_y$. Volume rendering algorithms are used to display the collected slice data with some type of transparency, and various techniques are used to accentuate the parts of the rendered volume that are of interest.

In this project, you’ll look at a volume rendering called *volume ray casting*, which takes full advantage of the graphics processing unit (GPU) to perform computations using OpenGL Shading Language

(GLSL) shaders. Your code executes for every pixel onscreen and leverages the GPU, which is designed to do parallel computations efficiently. You'll use a folder of 2D images consisting of slices from a 3D data set to construct a volume-rendered image using the volume ray casting algorithm. You'll also implement a method to show 2D slices of the data in the x-, y-, and z-directions so users can scroll through the slices using the arrow keys. Keyboard commands will let the user toggle between the 3D rendering and the 2D slices.

Here are some of the topics covered in this project:

- Using GLSL for GPU computations
- Creating vertex and fragment shaders
- Representing 3D volumetric data and using the volume ray casting algorithm
- Using `numpy` arrays for 3D transformation matrices

How It Works

There are various ways to render a 3D data set. In this project, you'll use the volume ray casting method, which is an *image-based* rendering technique used to generate the final image from the 2D slices, pixel by pixel. In contrast, typical 3D rendering methods are *object based*: they begin with a 3D object representation and then apply transformations to generate the pixels in the projected 2D image.

In the volume ray casting method that you'll use in this project, for each pixel in the output image, a ray is shot into the discrete 3D volumetric data set, which is typically represented as a cuboid. As the ray passes through the volume, the data is sampled at regular intervals, and the samples are combined, or *composited*, to compute the color value or intensity of the final image. (You might think of this process as similar to stacking a bunch of transparencies on top of each other and holding them up against a bright light to see a blend of all the sheets.)

While volume ray casting rendering implementations typically use techniques such as applying gradients to improve the appearance of the final render, filtering to isolate 3D features, and using spatial optimization techniques to improve speed, you'll just implement the basic ray casting algorithm and composite the final image by X-ray casting. (My implementation is largely based on the seminal paper on this topic by Kruger and Westermann, published in 2003.¹)

The Data Format

For this project, you'll use medical data from 3D scans from the Stanford Volume Data Archive.² This archive offers a few excellent 3D medical data sets (both CT and MRI) of TIFF images, one for each 2D cross section of the volume. You'll read a folder of these images into an OpenGL 3D texture; this is sort of like stacking a set of 2D images to form a cuboid, as shown in [Figure 11-1](#).

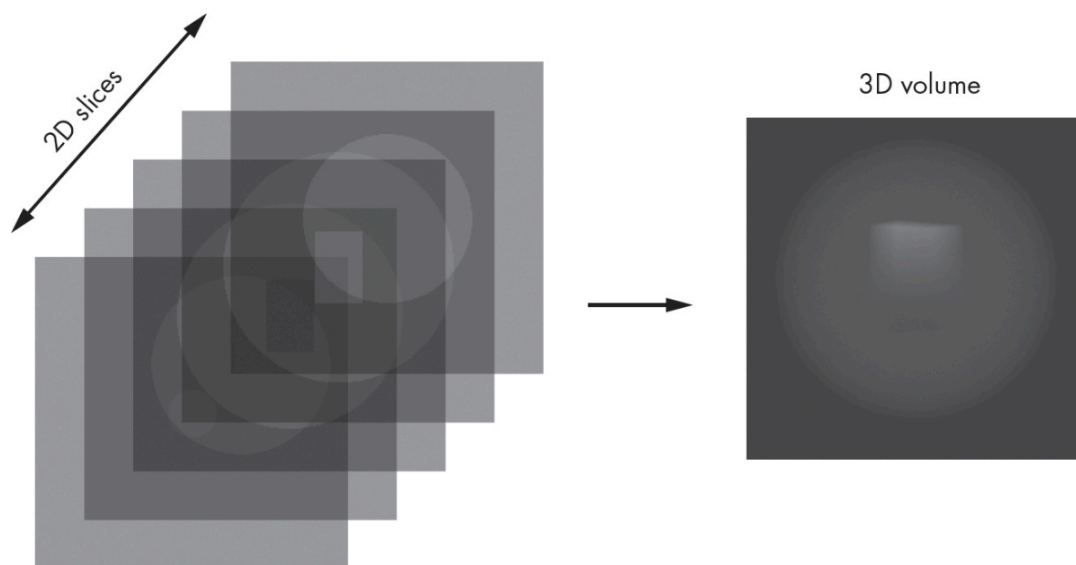


Figure 11-1: Building 3D volumetric data from 2D slices

Recall from [Chapter 9](#) that a 2D texture in OpenGL is addressed with a 2D coordinate (s, t) . Similarly, a 3D texture is addressed using a 3D texture coordinate of the form (s, t, p) . As you'll see, storing the volumetric data as a 3D texture allows you to access the data quickly and pro-

vides you with interpolated values required by your ray casting scheme.

Ray Generation

Your goal in this project is to generate a perspective projection of the 3D volumetric data, as shown in **Figure 11-2**. The figure shows the OpenGL view frustum, as discussed in **Chapter 9**. Specifically, it shows how a ray from the eye enters this frustum at the near plane, passes through the cubic volume (which contains the volumetric data), and exits from the rear at the far plane.

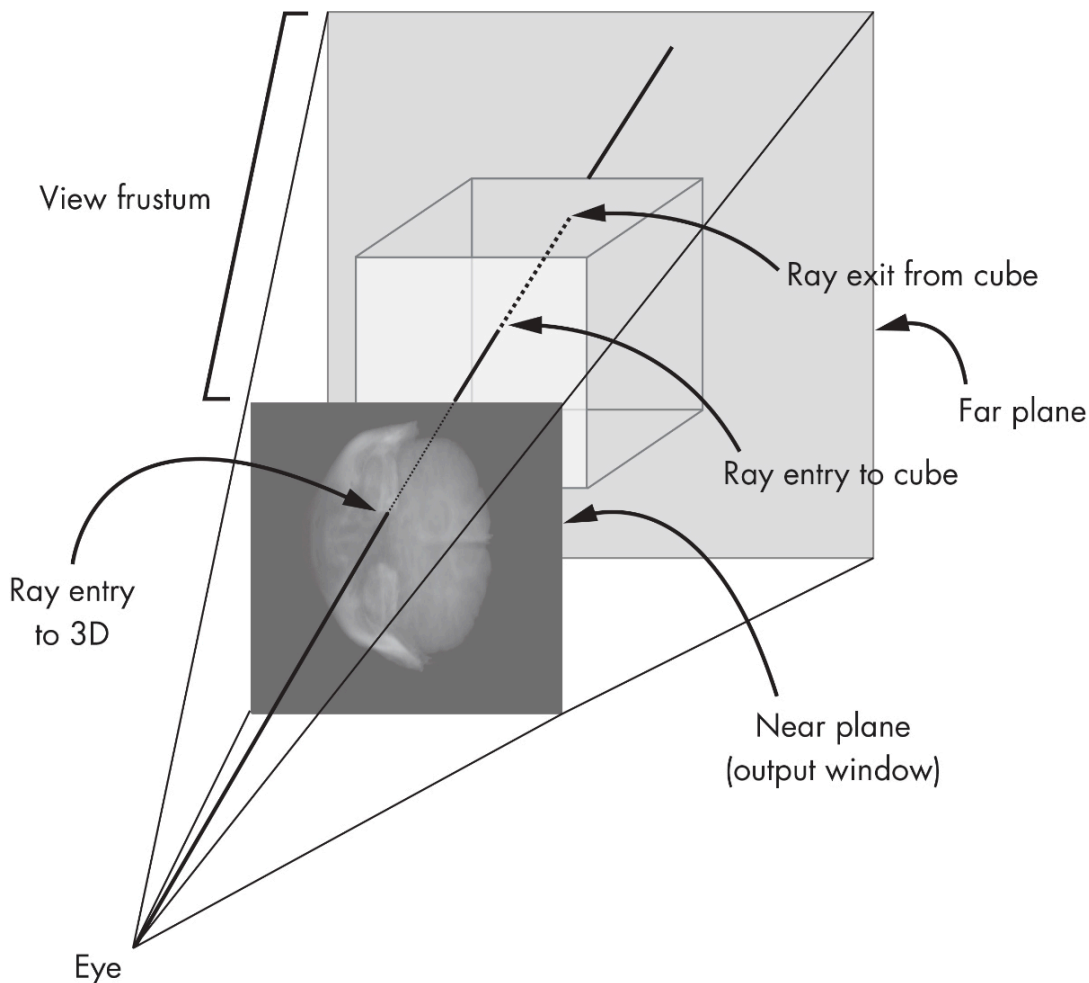


Figure 11-2: A perspective projection of 3D volumetric data

To implement ray casting, you need to generate rays that go into the volume. For each pixel in the output window shown in **Figure 11-2**,

you generate a vector R that goes into the volume you consider a unit cube (which I'll refer to as the *color cube*) defined between the coordinates (0, 0, 0) and (1, 1, 1). You color each point inside this cube with the RGB values equal to the 3D coordinates of the cube. The origin is colored (0, 0, 0), or black; the (1, 0, 0) corner is red; and the point on the cube diagonally opposite the origin is colored (1, 1, 1), or white.

Figure 11-3 shows this cube.

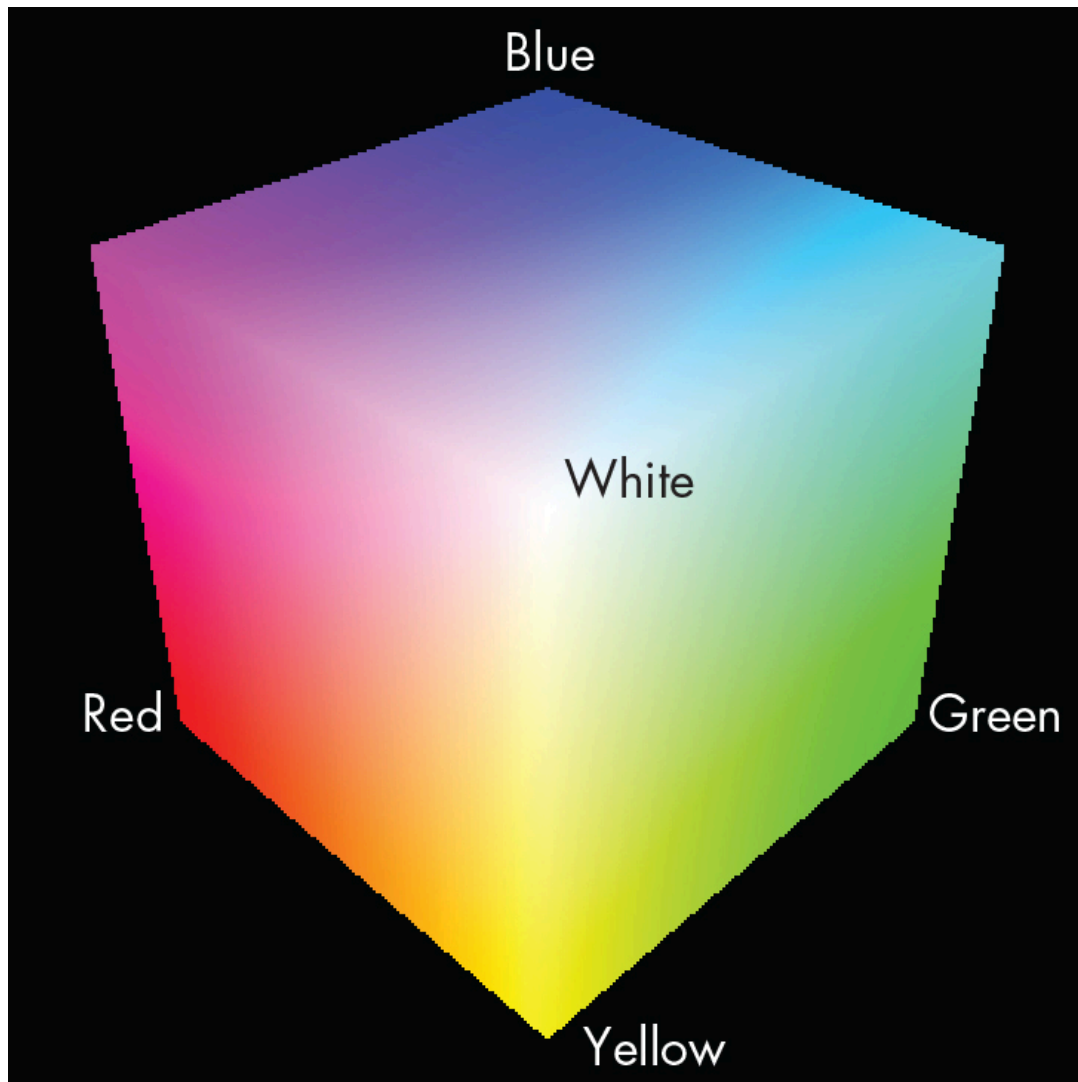


Figure 11-3: A color cube

NOTE In OpenGL, a color can be represented as a strip of 8-bit unsigned values (r, g, b), where r, g, and b are in the range [0, 255]. It can also be represented as a triplet of 32-bit floating-point values (r, g, b), where r, g, and b are in the range [0.0, 1.0]. These representations are equivalent.

For example, the red color (255, 0, 0) in the former is the same as (1.0, 0.0, 0.0) in the latter.

To draw the cube, first draw its six faces using the OpenGL primitive `GL_TRIANGLES`. Then color each vertex and use the interpolation provided by OpenGL when it rasterizes polygons to take care of the colors between each vertex. For example, **Figure 11-4(a)** shows the three front-faces of the cube. The back-faces of the cube are drawn in **Figure 11-4(b)** by setting OpenGL to cull front-faces.

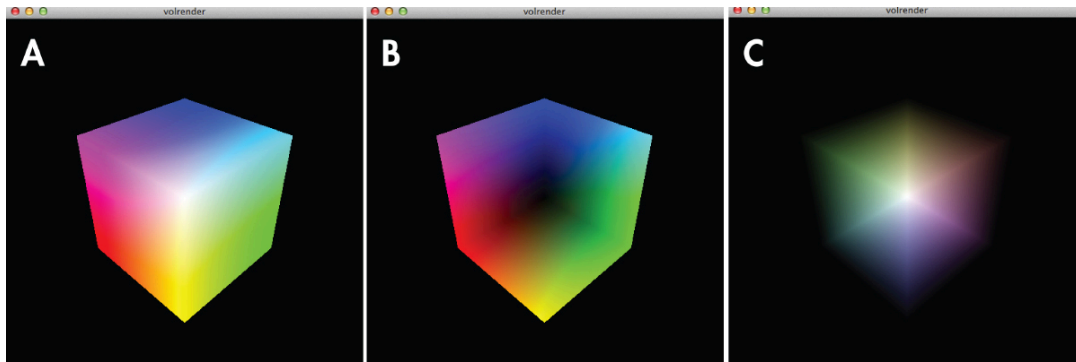


Figure 11-4: The color cube used to compute rays

If you subtract the colors in **Figure 11-4(a)** from **Figure 11-4(b)** by subtracting $(r, g, b)_{\text{front}}$ from $(r, g, b)_{\text{back}}$, you actually compute a set of vectors that go from the front to the back of the cube because each color (r, g, b) on this cube is the same as the 3D coordinate at that color's location. **Figure 11-4(c)** shows the result. (Negative values have been flipped to positive for the purposes of this illustration because negative numbers cannot be displayed as colors directly.) Reading the color value (r, g, b) of a pixel, as shown in **Figure 11-4(c)**, gives the (r_x, r_y, r_z) coordinates for the ray passing into the volume at that point.

Once you have the casting rays, you render them into an image or 2D texture for later use with OpenGL's frame buffer object (FBO) feature. After this texture is generated, you can access it inside the shaders that you'll use to implement the ray casting algorithm.

Ray Casting in the GPU

To implement the ray casting algorithm, you first draw the back-faces of the color cube into an FBO. Next, the front-faces are drawn on the screen. The bulk of the ray casting algorithm happens in the fragment shader for this second rendering, which runs for each pixel in the output. The ray is computed by subtracting the front-face color of the incoming fragment from the back-face color of the color cube, which is read in from a texture. The computed ray is then used to accumulate and compute the final pixel value using the 3D volumetric texture data, available within the shader.

Showing 2D Slices

In addition to the 3D rendering, you show 2D slices of the data by extracting the 2D cross section from the 3D data perpendicular to the x-, y-, or z-axis and applying that as a texture on a quad. Because you store the volume as a 3D texture, you can easily get the required data by specifying the texture coordinates (s , t , p). OpenGL's built-in texture interpolation gives you the texture values anywhere inside the 3D texture.

The OpenGL Window

As in your other OpenGL projects, this project uses the GLFW library to display the OpenGL window. You'll use handlers for drawing, for resizing the window, and for keyboard events. You'll use keyboard events to toggle between volume and slice rendering, as well as for rotating and slicing through the 3D data.

Requirements

You'll use `PyOpenGL`, a popular Python binding for OpenGL, for rendering. You'll also use the Python Imaging Library (PIL) to load the

2D images from the volumetric data set, and you'll use `numpy` arrays to represent 3D coordinates and transformation matrices.

The Code

You'll begin by generating a 3D texture from the volumetric data read in from the image files. Next, you'll look at a color cube technique for generating rays from the eye that point into the volume, which is a key concept in implementing the volume ray casting algorithm. You'll look at how to define the cube geometry as well as how to draw the back- and front-faces of this cube. You'll then explore the volume ray casting algorithm and the associated vertex and fragment shaders. Finally, you'll learn how to implement 2D slicing of the volumetric data.

This project has seven Python files:

glutils.py Contains the utility methods for OpenGL shaders, transformations, and so on

makedata.py Contains utility methods for creating volumetric data for testing

raycast.py Implements the `RayCastRender` class for ray casting

raycube.py Implements the `RayCube` class for use in `RayCastRender`

slicerender.py Implements the `SliceRender` class for 2D slicing of volumetric data

volreader.py Contains the utility method to read volumetric data into the OpenGL 3D texture

volrender.py Contains the main methods that create the GLFW window and the renderers

We'll cover all but two of these files in this chapter. The *makedata.py* file lives with the other project files for this chapter at <https://github.com/mkvenkit/pp2e/tree/main/volrender>. The *glutils.py* file can be downloaded from <https://github.com/mkvenkit/pp2e/tree/main/common>.

Generating a 3D Texture

The first step is to read the volumetric data from a folder containing images, as shown in the following code. To see the complete *volreader.py* code, skip ahead to [“The Complete 3D Texture Code”](#) on [page 241](#). You can also find the *volreader.py* file at <https://github.com/mkvenkit/pp2e/tree/main/volrender>. Note that the `loadTexture()` function in this file is used to open an image file, read the contents, and create an OpenGL texture object out of it, which is subsequently used in rendering.

```
def loadVolume(dirName):
```

```
    """read volume from directory as a 3D texture"""
```

```
    # list images in directory
```

```
    ❶ files = sorted(os.listdir(dirName))
```

```
    print('loading images from: %s' % dirName)
```

```
    imgDataList = []
```

```
    count = 0
```

```
    width, height = 0, 0
```

```
    for file in files:
```

```
        ❷ file_path = os.path.abspath(os.path.join(dirName, file))
```

try:

read image

❸ `img = Image.open(file_path)`

`imgData = np.array(img.getdata(), np.uint8)`

check if all images are of the same size

❹ if count is 0:

`width, height = img.size[0], img.size[1]`

`imgDataList.append(imgData)`

else:

❺ if `(width, height) == (img.size[0], img.size[1])`:

`imgDataList.append(imgData)`

else:

`print('mismatch')`

`raise RuntimeError("image size mismatch")`

`count += 1`

print `img.size`

❻ except:

skip

`print('Invalid image: %s' % file_path)`

```
# load image data into single array
```

```
depth = count
```

```
❶ data = np.concatenate(imgDataList)
```

```
print('volume data dims: %d %d %d' % (width, height, depth))
```

The `loadVolume()` method first lists the files in the given directory using the `listdir()` method from the `os` module ❶. Then you iterate through the image files themselves, loading them one at a time. To do this, you append the current filename to the directory using `os.path.abspath()` and `os.path.join()` ❷, eliminating the need to deal with relative filepaths and operating system–specific path conventions. (You often see this useful idiom in Python code that traverses files and directories.) Next, you use the `Image` class from the PIL to load the current image into an 8-bit `numpy` array ❸. If the file specified isn't an image or if the image fails to load, an exception is thrown, which you catch by printing an error ❹.

Because you're loading these image slices into a 3D texture, you need to ensure they all have the same dimensions (width × height), which you confirm at ❺ and ❻. You store the dimensions for the first image and compare them against new incoming images. Once all the images are loaded into individual arrays, you create the final array containing the 3D data by joining these arrays using the `concatenate()` method from `numpy` ❼.

The `loadVolume()` function continues by loading the array of 3D image data into an OpenGL texture:

```
# load data into 3D texture
```

```
❶ texture = glGenTextures(1)
```

```
glPixelStorei(GL_UNPACK_ALIGNMENT, 1)
```

```
glBindTexture(GL_TEXTURE_3D, texture)
```

```
glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_WRAP_S,  
GL_CLAMP_TO_EDGE)
```

```
glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_WRAP_T,  
GL_CLAMP_TO_EDGE)
```

```
glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_WRAP_R,  
GL_CLAMP_TO_EDGE)
```

```
glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_MAG_FILTER,  
GL_LINEAR)
```

```
glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_MIN_FILTER,  
GL_LINEAR)
```

```
❷ glTexImage3D(GL_TEXTURE_3D, 0, GL_RED,
```

```
width, height, depth, 0,
```

```
GL_RED, GL_UNSIGNED_BYTE, data)
```

```
# return texture
```

```
❸ return (texture, width, height, depth)
```

Here you create an OpenGL texture ❶ and set parameters for filtering and unpacking. Then you load the 3D data array into the OpenGL texture ❷. The format used here is `GL_RED`, and the data format is `GL_UNSIGNED_BYTE` because you have only one 8-bit value associated with each pixel in the data. Finally, you return the OpenGL texture ID and the dimensions of the 3D texture ❸.

Generating Rays

The code for generating the rays is encapsulated in a class called `RayCube`. This class is responsible for drawing the color cube and has methods to draw the back-faces of the cube to an FBO or texture and to draw the front-faces of the cube to the screen. To see the complete `raycube.py` code, skip ahead to [“The Complete Ray Generation Code”](#) on [page 242](#). You can also find the `raycube.py` file at <https://github.com/mkvenkit/pp2e/tree/main/volrender>.

First, let’s define the shaders used by this class. The shaders will be compiled as part of the `RayCube` class’s constructor:

```
❶ strVS = """

# version 410 core

layout(location = 1) in vec3 cubePos;

layout(location = 2) in vec3 cubeCol;

uniform mat4 uMVMatrix;

uniform mat4 uPMatrix;

out vec4 vColor;

void main()

{

    // set back-face color

    vColor = vec4(cubeCol.rgb, 1.0);

    // transformed position

    vec4 newPos = vec4(cubePos.xyz, 1.0);
```

```
// set position

gl_Position = uPMatrix * uMVMatrix * newPos;

}

"""

❷ strFS = """

# version 410 core

in vec4 vColor;

out vec4 fragColor;

void main()

{

    fragColor = vColor;

}

"""
```

You define the vertex shader used by the `RayCube` class ❶. This shader has two input attributes, `cubePos` and `cubeCol`, which are used to access the position and color values of the vertices, respectively. The modelview and projection matrices are passed in with the uniform variables `uMVMatrix` and `uPMatrix`, respectively. The `vColor` variable is declared as output because it needs to be passed on to the fragment shader, where it will be interpolated. The fragment shader ❷ sets the fragment color to the (interpolated) value of the incoming `vColor` set in the vertex shader.

Defining the Color Cube Geometry

Now let's look at the geometry of the color cube, defined in the `RayCube` class's constructor:

```
class RayCube:
```

```
    def __init__(self, width, height):
```

```
        -- snip --
```

```
        # cube vertices
```

```
        ❶ vertices = numpy.array([
```

```
            0.0, 0.0, 0.0,
```

```
            1.0, 0.0, 0.0,
```

```
            1.0, 1.0, 0.0,
```

```
            0.0, 1.0, 0.0,
```

```
            0.0, 0.0, 1.0,
```

```
            1.0, 0.0, 1.0,
```

```
            1.0, 1.0, 1.0,
```

```
            0.0, 1.0, 1.0
```

```
        ], numpy.float32)
```

```
        # cube colors
```

```
        ❷ colors = numpy.array([
```

```
            0.0, 0.0, 0.0,
```

```
1.0, 0.0, 0.0,  
  
1.0, 1.0, 0.0,  
  
0.0, 1.0, 0.0,  
  
0.0, 0.0, 1.0,  
  
1.0, 0.0, 1.0,  
  
1.0, 1.0, 1.0,  
  
0.0, 1.0, 1.0  
  
], numpy.float32)  
  
# individual triangles  
  
③ indices = numpy.array([  
  
    4, 5, 7,  
  
    7, 5, 6,  
  
    5, 1, 6,  
  
    6, 1, 2,  
  
    1, 0, 2,  
  
    2, 0, 3,  
  
    0, 4, 3,  
  
    3, 4, 7,  
  
    6, 2, 7,
```

7, 2, 3,

4, 0, 5,

5, 0, 1

], numpy.int16)

You define the cube geometry ❶ and the colors ❷ as `numpy` arrays. Notice that the values in these two definitions are the same. As we discussed earlier, the color of each pixel in the color cube corresponds to that pixel's 3D coordinates. The color cube has six faces, each of which can be drawn as two triangles, for a total of 6×6 , or 36, vertices. But rather than specify all 36 vertices, you specify just the cube's eight corners ❶ and then define the triangles formed with those corners using an `indices` array ❸, as illustrated in **Figure 11-5**. The first two sets of three indices, for example, (4, 5, 7) and (7, 5, 6), define the triangles on the top face of the cube.

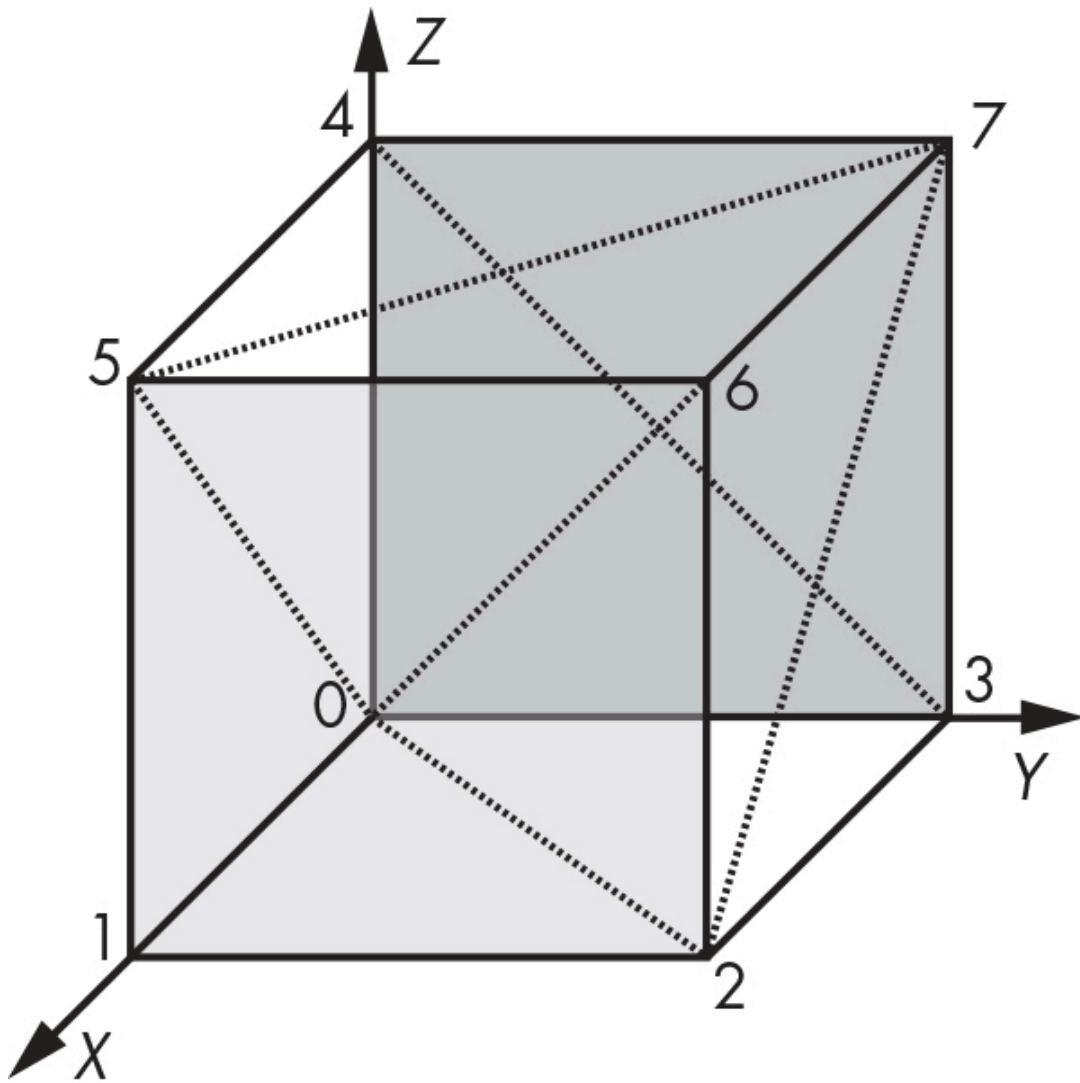


Figure 11-5: Using indexing, a cube can be represented as a collection of triangles, with each face composed of two triangles.

Next, still within the `RayCube` class constructor, you need to put the vertex information into buffers:

```
# set up vertex array object (VAO)
```

```
self.vao = glGenVertexArrays(1)
```

```
glBindVertexArray(self.vao)
```

```
# vertex buffer
```

```
self.vertexBuffer = glGenBuffers(1)
```

```

glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

glBufferData(GL_ARRAY_BUFFER, 4*len(vertices), vertices,
GL_STATIC_DRAW)

# vertex buffer – cube vertex colors

self.colorBuffer = glGenBuffers(1)

glBindBuffer(GL_ARRAY_BUFFER, self.colorBuffer)

glBufferData(GL_ARRAY_BUFFER, 4*len(colors), colors,
GL_STATIC_DRAW)

# index buffer

self.indexBuffer = glGenBuffers(1)

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, self.indexBuffer); ❶

glBufferData(GL_ELEMENT_ARRAY_BUFFER, 2*len(indices),
indices,

GL_STATIC_DRAW)

```

As with previous projects, you create and bind to a vertex array object (VAO) and then define the buffers it manages. One difference here is that the `indices` array is given the designation `GL_ELEMENT_ARRAY_BUFFER` ❶, which means the elements in its buffer will be used to index and access the data in the color and vertex buffers.

Creating the Frame Buffer Object

Now let's jump to the `RayCube` class method that creates the frame buffer object, where you'll direct your rendering:

```
def initFBO(self):

    # create frame buffer object

    self.fboHandle = glGenFramebuffers(1)

    # create texture

    self.texHandle = glGenTextures(1)

    # create depth buffer

    self.depthHandle = glGenRenderbuffers(1)

    # bind

    glBindFramebuffer(GL_FRAMEBUFFER, self.fboHandle)

    glActiveTexture(GL_TEXTURE0)

    glBindTexture(GL_TEXTURE_2D, self.texHandle)

    # set parameters to draw the image at different sizes

    ❶ glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR)

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR)

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE)

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE)

    # set up texture
```

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, self.width, self.height,  
  
             0, GL_RGBA, GL_UNSIGNED_BYTE, None)  
  
# bind texture to FBO  
  
❷ glFramebufferTexture2D(GL_FRAMEBUFFER,  
GL_COLOR_ATTACHMENT0,  
  
GL_TEXTURE_2D, self.texHandle, 0)  
  
# bind  
  
❸ glBindRenderbuffer(GL_RENDERBUFFER, self.depthHandle)  
  
glRenderbufferStorage(GL_RENDERBUFFER,  
GL_DEPTH_COMPONENT24,  
  
self.width, self.height)  
  
# bind depth buffer to FBO  
  
glFramebufferRenderbuffer(GL_FRAMEBUFFER,  
GL_DEPTH_ATTACHMENT,  
  
GL_RENDERBUFFER, self.depthHandle)  
  
# check status  
  
❹ status = glCheckFramebufferStatus(GL_FRAMEBUFFER)  
  
if status == GL_FRAMEBUFFER_COMPLETE:  
  
    pass  
  
# print "fbo %d complete" % self.fboHandle
```

```
elif status == GL_FRAMEBUFFER_UNSUPPORTED:
```

```
    print "fbo %d unsupported" % self.fboHandle
```

```
else:
```

```
    print "fbo %d Error" % self.fboHandle
```

Here you create a frame buffer object, a 2D texture, and a render buffer object; then you set up the texture parameters ❶. The texture is bound to the frame buffer ❷, and at ❸ and in the lines that follow, the render buffer sets up a 24-bit depth buffer and is attached to the frame buffer. Next, you check the status of the frame buffers ❹ and print a status message if something goes wrong. Now, as long as the frame buffer and render buffer are bound correctly, all of your rendering will go into the texture.

Rendering the Back-Faces of the Cube

Here is the code for rendering the back-faces of the color cube:

```
def renderBackFace(self, pMatrix, mvMatrix):
```

```
    """renders back-face of ray-cube to a texture and returns it"""
```

```
    # render to FBO
```

```
    ❶ glBindFramebuffer(GL_FRAMEBUFFER, self.fboHandle)
```

```
    # set active texture
```

```
    glActiveTexture(GL_TEXTURE0)
```

```
    # bind to FBO texture
```

```
    glBindTexture(GL_TEXTURE_2D, self.texHandle)
```

```
# render cube with face culling enabled
```

```
❷ self.renderCube(pMatrix, mvMatrix, self.program, True)
```

```
# unbind texture
```

```
❸ glBindTexture(GL_TEXTURE_2D, 0)
```

```
glBindFramebuffer(GL_FRAMEBUFFER, 0)
```

```
glBindRenderbuffer(GL_RENDERBUFFER, 0)
```

```
# return texture ID
```

```
❹ return self.texHandle
```

First you bind the FBO ❶, set the active texture unit, and bind to the texture handle so that you can render to the FBO. Then you call the `RayCube` class's `renderCube()` method ❷, which we'll look at soon. It has a face-culling flag as an argument, allowing you to draw either the front-face or the back-face of the cube using the same code. You set the flag to `True` to make the back-faces appear in the FBO texture.

Next, you make the necessary calls to unbind from the FBO so that other rendering code is unaffected ❸. Finally, you return the FBO texture ID ❹ for use in the next stage of the algorithm.

Rendering the Front-Faces of the Cube

The following code is used to draw the front-faces of the color cube during the second rendering pass of the ray casting algorithm:

```
def renderFrontFace(self, pMatrix, mvMatrix, program):
```

```
    """render front-face of ray-cube"""
```

```
# no face culling
```

```
self.renderCube(pMatrix, mvMatrix, program, False)
```

This method simply calls `renderCube()`, with the face-culling flag set to `False` so the front-faces will appear.

Rendering the Whole Cube

Now let's look at the `renderCube()` method, which draws the color cube discussed previously:

```
def renderCube(self, pMatrix, mvMatrix, program, cullFace):  
  
    """renderCube uses face culling if flag set"""  
  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
  
    # set shader program  
  
    glUseProgram(program)  
  
    # set projection matrix  
  
    glUniformMatrix4fv(glGetUniformLocation(program, b'uPMatrix'),  
                        1, GL_FALSE, pMatrix)  
  
    # set modelview matrix  
  
    glUniformMatrix4fv(glGetUniformLocation(program,  
b'uMVMMatrix'),  
                        1, GL_FALSE, mvMatrix)  
  
    # enable face culling
```

```
glDisable(GL_CULL_FACE)
```

❶ if cullFace:

```
    glFrontFace(GL_CCW)
```

```
    glCullFace(GL_FRONT)
```

```
    glEnable(GL_CULL_FACE)
```

```
# bind VAO
```

```
glBindVertexArray(self.vao)
```

```
# animated slice
```

❷ glDrawElements(GL_TRIANGLES, self.nIndices,
GL_UNSIGNED_SHORT, None)

```
# unbind VAO
```

```
glBindVertexArray(0)
```

```
# reset cull face
```

```
if cullFace:
```

```
    # disable face culling
```

```
    glDisable(GL_CULL_FACE)
```

You clear the color and depth buffers, select the shader program, and set the transformation matrices. Then you set a flag to control face culling ❶, which determines whether the cube's front-face or back-face is drawn. Notice that you use `glDrawElements()` ❷ because you're using an index array to render the cube, rather than a vertex array.

Resizing the Window

Because the FBO is created for a particular window size, you need to re-create it when the window size changes. To do that, you create a resize handler for the `RayCube` class, as shown here:

```
def reshape(self, width, height):

    self.width = width

    self.height = height

    self.aspect = width/float(height)

    # re-create FBO

    self.clearFBO()

    self.initFBO()
```

The `reshape()` method is called when the OpenGL window is resized. It checks the new window dimensions and then clears and re-creates the FBO.

Implementing the Ray Casting Algorithm

Next, you'll implement the ray casting algorithm in the `RayCastRender` class. The core of the algorithm happens inside the fragment shader used by this class, which also uses the `RayCube` class to help generate the rays. To see the complete *raycast.py* code, skip ahead to **"The Complete Volume Ray Casting Code"** on **page 248**. You can also find this file at

<https://github.com/mkvenkit/pp2e/tree/main/volrender>.

Begin in the `RayCastRender` constructor by creating a `RayCube` object and loading the shaders:

```
class RayCastRender:

    def __init__(self, width, height, volume):

        """RayCastRender construction"""

        # create RayCube object

        ❶ self.raycube = raycube.RayCube(width, height)

        # set dimensions

        self.width = width

        self.height = height

        self.aspect = width/float(height)

        # create shader

        ❷ self.program = glutils.loadShaders(strVS, strFS)

        # texture

        ❸ self.texVolume, self.Nx, self.Ny, self.Nz = volume

        # initialize camera

        ❹ self.camera = Camera()
```

The constructor creates an object of type `RayCube` ❶, which is used to generate rays. You load the shaders used by the ray casting ❷ and then set the OpenGL 3D texture and dimensions ❸, which were passed into the constructor as the tuple `volume`. Next, you create a `Camera` object ❹, which you'll use to set up the OpenGL perspective transformation for the 3D rendering.

NOTE The `Camera` class, also declared in `raycast.py`, is basically the same as the one used in [Chapter 10](#). You'll see it in the complete code listing on [page 248](#).

Here is the rendering method for `RayCastRender` :

```
def draw(self):

    # build projection matrix

    ❶ pMatrix = glutils.perspective(45.0, self.aspect, 0.1, 100.0)

    # modelview matrix

    ❷ mvMatrix = glutils.lookAt(self.camera.eye, self.camera.center,

                                self.camera.up)

    # render

    # generate ray-cube back-face texture

    ❸ texture = self.raycube.renderBackFace(pMatrix, mvMatrix)

    # set shader program

    ❹ glUseProgram(self.program)

    # set window dimensions

    glUniform2f(glGetUniformLocation(self.program, b"uWinDims"),

                float(self.width), float(self.height))

    # bind to texture unit 0, which represents back-faces of cube

    ❺ glActiveTexture(GL_TEXTURE0)
```

```

glBindTexture(GL_TEXTURE_2D, texture)

glUniform1i(glGetUniformLocation(self.program, b"texBackFaces"),
0)

# texture unit 1: 3D volume texture

❸ glBindTexture(GL_TEXTURE_3D, self.texVolume)

glUniform1i(glGetUniformLocation(self.program, b"texVolume"), 1)

# draw front-face of cubes

❹ self.raycube.renderFrontFace(pMatrix, mvMatrix, self.program)

```

First you set up a perspective projection matrix for the rendering, using the `glutils.perspective()` utility method ❶. Then you set the current camera parameters into the `glutils.lookAt()` method ❷. Next, the first pass of the rendering is done ❸, using the `renderBackFace()` method in `RayCube` to draw the back-faces of the color cube into a texture. (This method also returns the ID of the generated texture.)

You continue by enabling the shaders for the ray casting algorithm ❹. Then you set up the textures for use in the shader program. The texture returned at ❸ is set up as texture unit 0 ❺, and the 3D texture created from the volumetric data you read in is set up as texture unit 1 ❻. Finally, you render the front-faces of the cube using the `renderFrontFace()` method in `RayCube` ❼. When this code is executed, the shaders for `RayCastRender` will act on the vertices and fragments.

The Vertex Shader

Now you come to the shaders used by `RayCastRender`. Let's look at the vertex shader first:

```
strVS = ""

# version 410 core

❶ layout(location = 1) in vec3 cubePos;

layout(location = 2) in vec3 cubeCol;

❷ uniform mat4 uMVMatrix;

uniform mat4 uPMatrix;

❸ out vec4 vColor;

void main()

{

    // set position

    ❹ gl_Position = uPMatrix * uMVMatrix * vec4(cubePos.xyz, 1.0);

    // set color

    ❺ vColor = vec4(cubeCol.rgb, 1.0);

}

""
```

First you set the input variables of position and color ❶. The layout uses the same indices as defined in the `RayCube` vertex shader because `RayCastRender` uses the VBO defined in that class to draw the geometry, and the locations in the shaders have to match. Then you

define the input transformation matrices ❷ and set a color value as the shader output ❸. The usual transformation computes the built-in `gl_Position` output ❹, before you set the output as the current color of the cube vertex ❺. The latter will be interpolated across vertices to give you the correct color in the fragment shader.

The Fragment Shader

The fragment shader is the star of the show. It implements the core of the ray casting algorithm.

```
strFS = ""

# version 410 core

in vec4 vColor;

uniform sampler2D texBackFaces;

uniform sampler3D texVolume;

uniform vec2 uWinDims;

out vec4 fragColor;

void main()

{

    // start of ray

    ❶ vec3 start = vColor.rgb;

    // calculate texture coordinates at fragment,

    // which is a fraction of window coordinates
```

```
❷ vec2 texc = gl_FragCoord.xy/uWinDims.xy;

// get end of ray by looking up back-face color

❸ vec3 end = texture(texBackFaces, texc).rgb;

// calculate ray direction

❹ vec3 dir = end - start;

// normalized ray direction

vec3 norm_dir = normalize(dir);

// the length from front to back is calculated and

// used to terminate the ray

float len = length(dir.xyz);

// ray step size

float stepSize = 0.01;

// X-ray projection

vec4 dst = vec4(0.0);

// step through the ray

❺ for(float t = 0.0; t < len; t += stepSize) {

    // set position to endpoint of ray

    ❻ vec3 samplePos = start + t*norm_dir;

    // get texture value at position
```

```

❷ float val = texture(texVolume, samplePos).r;

vec4 src = vec4(val);

// set opacity

❸ src.a *= 0.1;

src.rgb *= src.a;

// blend with previous value

❹ dst = (1.0 - dst.a)*src + dst;

// exit loop when alpha exceeds threshold

❺ if(dst.a >= 0.95)

    break;

}

// set fragment color

fragColor = dst;

}

"""

```

The input to the fragment shader is the cube vertex color. The fragment shader also has access to the 2D texture generated by rendering the color cube, the 3D texture containing the volumetric data, and the dimensions of the OpenGL window.

While the fragment shader executes, you send in the front-faces of the cube, so by looking up the incoming color value ❶, you get the starting

point of the ray that goes into this cube. (Recall the discussion in **“Ray Generation”** on **page 217** about the connection between the colors in the cube and the ray directions.)

You calculate the texture coordinate of the incoming fragment on the screen ❷. Here, dividing the location of the fragment in window coordinates by the window dimensions maps the location to the range [0, 1]. The ending point of the ray is obtained by looking up the back-face color of the cube using this texture coordinate ❸.

You next calculate the ray direction ❹ and then calculate the normalized direction and length of this ray, which will be useful in the ray casting computation. Then you loop through the volume using the ray’s starting point and direction until it hits the ray’s endpoint ❺. In this loop, you compute the ray’s current position inside the data volume ❻ and look up the data value at this point ❼. You then perform the blending equation at ❽ and ❾, which gives you the X-ray effect. You combine the `dst` value with the current value of the intensity (which is attenuated using the alpha value), and the process continues along the ray. The alpha value keeps increasing until it equals the maximum threshold of 0.95 ❿, at which point you exit the loop. The end result is a sort of average opacity through the volume at each pixel, which produces a “see-through” or X-ray effect. (Try varying the threshold and alpha attenuation to produce different effects.)

Showing 2D Slices

In addition to showing the 3D view of the volumetric data, you want to show 2D slices of the data in the x-, y-, and z-directions onscreen. The code for this is encapsulated in a class called `SliceRender`, which creates 2D volumetric slices. To see the complete *slicerender.py* code, skip ahead to **“The Complete 2D Slicing Code”** on **page 251**. You can also find the *slicerender.py* file at

<https://github.com/mkvenkit/pp2e/tree/main/volrender>.

Here's the initialization code in the `SliceRender` class constructor that sets up the geometry for the slices:

```
class SliceRender:
```

```
    def __init__(self, width, height, volume):
```

```
        -- snip --
```

```
        # set up vertex array object (VAO)
```

```
        self.vao = glGenVertexArrays(1)
```

```
        glBindVertexArray(self.vao)
```

```
        # define quad vertices
```

```
        ❶ vertexData = numpy.array([0.0, 1.0, 0.0,
                                   0.0, 0.0, 0.0,
                                   1.0, 1.0, 0.0,
                                   1.0, 0.0, 0.0], numpy.float32)
```

```
        # vertex buffer
```

```
        self.vertexBuffer = glGenBuffers(1)
```

```
        glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)
```

```
        glBufferData(GL_ARRAY_BUFFER, 4*len(vertexData), vertexData,
                     GL_STATIC_DRAW)
```

```
        # enable arrays
```

```
glEnableVertexAttribArray(self.vertIndex)

# set buffers

glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

glVertexAttribPointer(self.vertIndex, 3, GL_FLOAT, GL_FALSE, 0,
None)

# unbind VAO

glBindVertexArray(0)
```

This code sets up a VAO to manage the VBO, as in earlier examples. You define the geometry of a square in the XY plane ❶. (The vertex order is that of the `GL_TRIANGLE_STRIP`, introduced in [Chapter 9](#).) You'll use this same geometry regardless of whether you're showing slices perpendicular to *x*, *y*, or *z*. All that changes between these cases is the data plane that you pick to display from within the 3D texture. We'll return to this idea when we look at the vertex shader.

Here's a method to render the 2D slices:

```
def draw(self):

    # clear buffers

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # build projection matrix

    ❶ pMatrix = glutils.ortho(-0.6, 0.6, -0.6, 0.6, 0.1, 100.0)

    # modelview matrix

    ❷ mvMatrix = numpy.array([1.0, 0.0, 0.0, 0.0,
```

```
        0.0, 1.0, 0.0, 0.0,

        0.0, 0.0, 1.0, 0.0,

        -0.5, -0.5, -1.0, 1.0], numpy.float32)

# use shader

glUseProgram(self.program)

# set projection matrix

glUniformMatrix4fv(self.pMatrixUniform, 1, GL_FALSE, pMatrix)

# set modelview matrix

glUniformMatrix4fv(self.mvMatrixUniform, 1, GL_FALSE, mvMatrix)

# set current slice fraction

❸ glUniform1f(glGetUniformLocation(self.program, b"uSliceFrac"),

               float(self.currSliceIndex)/float(self.currSliceMax))

# set current slice mode

❹ glUniform1i(glGetUniformLocation(self.program, b"uSliceMode"),

               self.mode)

# enable texture

glActiveTexture(GL_TEXTURE0)

glBindTexture(GL_TEXTURE_3D, self.texture)

glUniform1i(glGetUniformLocation(self.program, b"tex"), 0)
```

```
# bind VAO

glBindVertexArray(self.vao)

# draw

glDrawArrays(GL_TRIANGLE_STRIP, 0, 4)

# unbind VAO

glBindVertexArray(0)
```

Each 2D slice is a square, which you build up using an OpenGL triangle strip primitive. This code goes through the render setup for the triangle strip. Note that you implement the orthographic projection using the `glutils.ortho()` method at ❶. You set up a projection that adds a 0.1 buffer around the unit square representing the slice.

When you draw something with OpenGL, the default view (without any transformation applied) puts the eye at (0, 0, 0) looking down the z-axis with the y-axis pointing up. Applying the translation (-0.5, -0.5, -1.0) to your geometry centers it on the z-axis ❷. You set the current slice fraction ❸ (where, for example, the 10th slice out of 100 would be 0.1), set the slice mode ❹ (to view the slices in the x-, y-, or z-direction, as represented by the integers 0, 1, and 2, respectively), and set both values to the shaders.

The Vertex Shader

Now let's look at the vertex shader for `SliceRender`:

```
strVS = """

# version 410 core

in vec3 aVert;
```

```
uniform mat4 uMVMMatrix;

uniform mat4 uPMatrix;

uniform float uSliceFrac;

uniform int uSliceMode;

out vec3 texcoord;

void main() {

    // x slice

    if (uSliceMode == 0) {

        ❶ texcoord = vec3(uSliceFrac, aVert.x, 1.0-aVert.y);

    }

    // y slice

    else if (uSliceMode == 1) {

        ❷ texcoord = vec3(aVert.x, uSliceFrac, 1.0-aVert.y);

    }

    // z slice

    else {

        ❸ texcoord = vec3(aVert.x, 1.0-aVert.y, uSliceFrac);

    }

    // calculate transformed vertex
```

```

gl_Position = uPMatrix * uMVMatrix * vec4(aVert, 1.0);

}

"""

```

The vertex shader takes the triangle strip vertex array as input and sets a texture coordinate as output. The current slice fraction and slice mode are passed in as uniform variables `uSliceFrac` and `uSliceMode`.

The shader has three branches, depending on the slice mode. For example, if `uSliceMode` is `0`, you calculate the texture coordinates for an x slice ❶. Because you're slicing perpendicular to the x -direction, you want a slice parallel to the YZ plane. The 3D vertices coming in to the vertex shader also double as the 3D texture coordinates because they are in the range $[0, 1]$, so the texture coordinates are given as (f, V_x, V_y) , where f is the fraction of the slice number in the direction of the x -axis and where V_x and V_y are the vertex coordinates.

Unfortunately, the resulting image will appear upside down because the OpenGL coordinate system has its origin at the bottom left, with the y -direction pointing up; this is the reverse of what you want. To resolve this problem, you change the texture coordinate t to $(1 - t)$ and use $(f, V_x, 1 - V_y)$ ❷. You use similar logic to compute the texture coordinates for y - ❸ and z - ❹ direction slices if the `uSliceMode` value is `1` or `2`, respectively.

The Fragment Shader

Here is the fragment shader:

```

strFS = """

# version 410 core

❶ in vec3 texcoord;

```

```

❷ uniform sampler3D texture;

out vec4 fragColor;

void main() {

    // look up color in texture

    ❸ vec4 col = texture(tex, texcoord);

    ❹ fragColor = col.rrra;

}

"""

```

The fragment shader declares `texcoord` as input ❶, which was set as output in the vertex shader. The texture sampler is declared as `uniform` ❷. You look up the texture color using `texcoord` ❸ and set `fragColor` as the output ❹. (Because you read in your texture only as the red channel, you use `col.rrra`.)

A User Interface for 2D Slicing

Now you need a way for the user to slice through the data. Do this using a keyboard handling method on the `SliceRender` class:

```

def keyPressed(self, key):

    """keypress handler"""

    ❶ if key == 'x':

        self.mode = SliceRender.XSLICE

        # reset slice index

```

```
self.currSliceIndex = int(self.Nx/2)

self.currSliceMax = self.Nx

elif key == 'y':

    self.mode = SliceRender.YSLICE

    # reset slice index

    self.currSliceIndex = int(self.Ny/2)

    self.currSliceMax = self.Ny

elif key == 'z':

    self.mode = SliceRender.ZSLICE

    # reset slice index

    self.currSliceIndex = int(self.Nz/2)

    self.currSliceMax = self.Nz

elif key == 'l':

    ❷ self.currSliceIndex = (self.currSliceIndex + 1) % self.currSliceMax

elif key == 'r':

    self.currSliceIndex = (self.currSliceIndex - 1) % self.currSliceMax
```

When the X, Y, or Z key on the keyboard is pressed, `SliceRender` switches to the x, y, or z slice mode. You can see this in action for the x slice, for example ❶, where you set the appropriate mode, set the current slice index to the middle of the data, and update the maximum slice number.

When the left or right arrow key on the keyboard is pressed, you page through the slices. For instance, the slice index is incremented ❷ when the left arrow key is pressed. The modulo operator (%) ensures that the index “rolls over” to 0 when you exceed the maximum value.

Putting the Code Together

Let’s take a quick look at the main file in the project, *volrender.py*. This file uses a class `RenderWin`, which creates and manages the GLFW OpenGL window. (I won’t cover this class in detail because it’s similar to the class used in [Chapters 9](#) and [10](#).) To see the complete *volrender.py* code, skip ahead to [“The Complete Main File Code”](#) on [page 254](#). You can also find the *volrender.py* file at <https://github.com/mkvenkit/pp2e/tree/main/volrender>.

In the initialization code for this class, you create the renderer as follows:

```
class RenderWin:
```

```
    def __init__(self, imageDir):
```

```
        -- snip --
```

```
        # load volume data
```

```
        ❶ self.volume = volreader.loadVolume(imageDir)
```

```
        # create renderer
```

```
        ❷ self.renderer = RayCastRender(self.width, self.height,
self.volume)
```

Here you read the 3D data into an OpenGL texture using the `loadVolume()` function we discussed earlier ❶. Then you create an object of type `RayCastRender` to display the data ❷.

The Keypress Handler

The `RenderWindow` class needs its own keyboard handler method for toggling between volume and slice rendering and for closing the window. This method also passes along keypresses to the `RayCastRender` and `SliceRender` classes' keyboard handlers, to either rotate the camera or navigate through the 2D slices.

```
def onKeyboard(self, win, key, scancode, action, mods):

    # print 'keyboard: ', win, key, scancode, action, mods

    # ESC to quit

    if key is glfw.GLFW_KEY_ESCAPE:

        self.renderer.close()

        self.exitNow = True

    else:

        ❶ if action is glfw.GLFW_PRESS or action is glfw.GLFW_REPEAT:

            if key == glfw.GLFW_KEY_V:

                # toggle render mode

                ❷ if isinstance(self.renderer, RayCastRender):

                    self.renderer = SliceRender(self.width, self.height,

                                                self.volume)

            else:

                self.renderer = RayCastRender(self.width, self.height,
```

```

        self.volume)

    # call reshape on renderer

    self.renderer.reshape(self.width, self.height)

else:

    # send keypress to renderer

    ❸ keyDict = {glfw.GLFW_KEY_X: 'x', glfw.GLFW_KEY_Y: 'y',

                 glfw.GLFW_KEY_Z: 'z', glfw.GLFW_KEY_LEFT: 'l',

                 glfw.GLFW_KEY_RIGHT: 'r'}

    try:

        self.renderer.keyPressed(keyDict[key])

    except:

        pass

```

Pressing ESC quits the program. You set other keypresses to work whether you have just pressed the key down or you are keeping it pressed ❶. If the V key is pressed, you toggle the renderer between volume and slice ❷, using Python's `isinstance()` method to identify the current class type. To handle any other keypress events (X, Y, Z, or the left and right arrows), you use a dictionary ❸ and pass the keypress to the current renderer's `keyPressed()` handler method. We looked at the slice renderer's `keyPressed()` method in [“A User Interface for 2D Slicing”](#) on [page 237](#), for example.

NOTE *I'm choosing not to pass in the `glfw.KEY` values directly and using a dictionary to convert these to character values instead, because it's*

a good practice to reduce dependencies in source files. Currently, the only file in this project that depends on GLFW is volrender.py. If you were to pass GLFW-specific types into other code, they would need to import and depend on the GLFW library too. Then, if you were to switch to a different OpenGL windowing toolkit, the code would become messy.

Running the Program

Here is a sample run of the application using data from the Stanford Volume Data Archive:

```
$ python volrender.py --dir mrbrain-8bit/
```

You should see something like **Figure 11-6**.

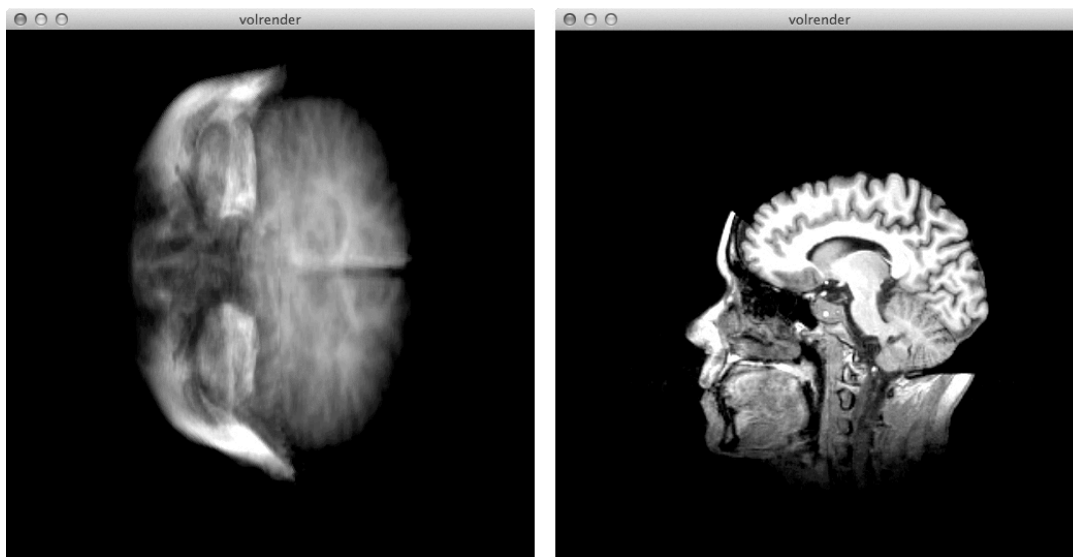


Figure 11-6: A sample run of volrender.py. The image on the left is the volumetric rendering, and the image on the right is a 2D slice.

As the application runs, use the V key to toggle between volume and slice rendering. In slice mode, use the X, Y, and Z keys to change the slicing axis, and use the arrow keys to change the slicing location.

Summary

In this chapter, you implemented the volume ray casting algorithm using Python and OpenGL. You learned how to use GLSL shaders to implement this algorithm efficiently, as well as how to create 2D slices from the volumetric data.

Experiments!

Here are a few ways you could keep tinkering with the volume ray casting program:

1. Currently, it's hard to see the boundary of the volumetric data "cube" in the ray casting mode. Implement a class `WireFrame` that draws a box around this cube. Color the x-, y-, and z-axes red, green, and blue, respectively, and give each its own shaders. You'll use `WireFrame` from within the `RayCastRender` class.
2. Implement data scaling. In the current implementation, you're drawing a cube for the volume and a square for 2D slices, which assumes you have a symmetric data set (that the number of slices are the same in each direction), but most real data has a varying number of slices. Medical data, in particular, often has fewer slices in the z-direction, with dimensions such as 256×256×99, for example. To display this data correctly, you have to introduce a scale into your computations. One way to do so is to apply the scale to the cube vertices (3D volume) and square vertices (2D slice). The user can then input the scaling parameters as command line arguments.
3. Our volume ray casting implementation uses X-ray casting to calculate the final color or intensity of a pixel. Another popular way to do this is to use *maximum intensity projection (MIP)* to set the maximum intensity at each pixel. Implement this in your code. (Hint: in the fragment shader of `RayCastRender`, modify the code that steps through the ray to check and set the maximum value along the ray, instead of blending values.)
4. Currently, the only UI you have implemented is rotation around the x-, y-, and z-axes. Implement a zoom feature so pressing I/O will zoom in/out of the volume-rendered image. You could do this by

setting the appropriate camera parameters in the `glutils.lookAt()` method, with one caveat: if you move your view inside the data cube, the ray casting will fail because OpenGL will clip the front-faces of the cube; the ray computation needed for ray casting requires both the front- and back-faces of the color cube to be rendered correctly. Instead, zoom by adjusting the field of view in the `glutils.projecton()` method.

The Complete 3D Texture Code

Here's the full *volreader.py* code listing.

```
import os

import numpy as np

from PIL import Image

import OpenGL

from OpenGL.GL import *

from scipy import misc

def loadVolume(dirName):

    """read volume from directory as a 3D texture"""

    # list images in directory

    files = sorted(os.listdir(dirName))

    print('loading images from: %s' % dirName)

    imgDataList = []
```

```
count = 0

width, height = 0, 0

for file in files:

    file_path = os.path.abspath(os.path.join(dirName, file))

    try:

        # read image

        img = Image.open(file_path)

        imgData = np.array(img.getdata(), np.uint8)

        # check if all are of the same size

        if count is 0:

            width, height = img.size[0], img.size[1]

            imgDataList.append(imgData)

        else:

            if (width, height) == (img.size[0], img.size[1]):

                imgDataList.append(imgData)

            else:

                print('mismatch')

                raise RuntimeError("image size mismatch")

        count += 1
```

```
# print img.size

except:

    # skip

    print('Invalid image: %s' % file_path)

# load image data into single array

depth = count

data = np.concatenate(imgDataList)

print('volume data dims: %d %d %d' % (width, height, depth))

# load data into 3D texture

texture = glGenTextures(1)

glPixelStorei(GL_UNPACK_ALIGNMENT, 1)

glBindTexture(GL_TEXTURE_3D, texture)

glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE)

glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE)

glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_WRAP_R,
GL_CLAMP_TO_EDGE)

glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR)
```

```
glTexParameteri(GL_TEXTURE_3D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR)

glTexImage3D(GL_TEXTURE_3D, 0, GL_RED,

            width, height, depth, 0,

            GL_RED, GL_UNSIGNED_BYTE, data)

# return texture

return (texture, width, height, depth)

# load texture

def loadTexture(filename):

    img = Image.open(filename)

    img_data = np.array(list(img.getdata()), 'B')

    texture = glGenTextures(1)

    glPixelStorei(GL_UNPACK_ALIGNMENT,1)

    glBindTexture(GL_TEXTURE_2D, texture)

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_CLAMP_TO_EDGE)

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_CLAMP_TO_EDGE)

    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR)
```

```
glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
GL_LINEAR)  
  
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, img.size[0],  
img.size[1],  
  
0, GL_RGBA, GL_UNSIGNED_BYTE, img_data)  
  
return texture
```

The Complete Ray Generation Code

Here's the full code listing for the `RayCube` class.

```
import OpenGL  
  
from OpenGL.GL import *  
  
from OpenGL.GL.shaders import *  
  
import numpy, math, sys  
  
import volreader, glutils  
  
strVS = ""  
  
# version 330 core  
  
layout(location = 1) in vec3 cubePos;  
  
layout(location = 2) in vec3 cubeCol;  
  
uniform mat4 uMVMatrix;  
  
uniform mat4 uPMatrix;
```

```
out vec4 vColor;

void main()

{

    // set back-face color

    vColor = vec4(cubeCol.rgb, 1.0);

    // transformed position

    vec4 newPos = vec4(cubePos.xyz, 1.0);

    // set position

    gl_Position = uPMatrix * uMVMatrix * newPos;

}

"""

strFS = """

# version 330 core

in vec4 vColor;

out vec4 fragColor;

void main()

{

    fragColor = vColor;

}
```

```
"""
```

```
class RayCube:
```

```
    """class used to generate rays used in ray casting"""
```

```
    def __init__(self, width, height):
```

```
        """RayCube constructor"""
```

```
        # set dims
```

```
        self.width, self.height = width, height
```

```
        # create shader
```

```
        self.program = glutils.loadShaders(strVS, strFS)
```

```
        # cube vertices
```

```
        vertices = numpy.array([
```

```
            0.0, 0.0, 0.0,
```

```
            1.0, 0.0, 0.0,
```

```
            1.0, 1.0, 0.0,
```

```
            0.0, 1.0, 0.0,
```

```
            0.0, 0.0, 1.0,
```

```
            1.0, 0.0, 1.0,
```

```
            1.0, 1.0, 1.0,
```

```
            0.0, 1.0, 1.0
```

```
    ], numpy.float32)

# cube colors

colors = numpy.array([

    0.0, 0.0, 0.0,

    1.0, 0.0, 0.0,

    1.0, 1.0, 0.0,

    0.0, 1.0, 0.0,

    0.0, 0.0, 1.0,

    1.0, 0.0, 1.0,

    1.0, 1.0, 1.0,

    0.0, 1.0, 1.0

], numpy.float32)

# individual triangles

indices = numpy.array([

    4, 5, 7,

    7, 5, 6,

    5, 1, 6,

    6, 1, 2,

    1, 0, 2,
```

```
2, 0, 3,  
  
0, 4, 3,  
  
3, 4, 7,  
  
6, 2, 7,  
  
7, 2, 3,  
  
4, 0, 5,  
  
5, 0, 1  
  
], numpy.int16)  
  
self.nIndices = indices.size  
  
# set up vertex array object (VAO)  
  
self.vao = glGenVertexArrays(1)  
  
glBindVertexArray(self.vao)  
  
# vertex buffer  
  
self.vertexBuffer = glGenBuffers(1)  
  
glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)  
  
glBufferData(GL_ARRAY_BUFFER, 4*len(vertices), vertices,  
GL_STATIC_DRAW)  
  
# vertex buffer - cube vertex colors  
  
self.colorBuffer = glGenBuffers(1)
```

```
glBindBuffer(GL_ARRAY_BUFFER, self.colorBuffer)

glBufferData(GL_ARRAY_BUFFER, 4*len(colors), colors,
GL_STATIC_DRAW);

# index buffer

self.indexBuffer = glGenBuffers(1)

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, self.indexBuffer);

glBufferData(GL_ELEMENT_ARRAY_BUFFER, 2*len(indices),
indices,

            GL_STATIC_DRAW)

# enable attrs using the layout indices in shader

aPosLoc = 1

aColorLoc = 2

# bind buffers

glEnableVertexAttribArray(1)

glEnableVertexAttribArray(2)

# vertex

glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

glVertexAttribPointer(aPosLoc, 3, GL_FLOAT, GL_FALSE, 0, None)

# color

glBindBuffer(GL_ARRAY_BUFFER, self.colorBuffer)
```

```
glVertexAttribPointer(aColorLoc, 3, GL_FLOAT, GL_FALSE, 0,
None)

# index

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, self.indexBuffer)

# unbind VAO

glBindVertexArray(0)

# FBO

self.initFBO()

def renderBackFace(self, pMatrix, mvMatrix):

    """renders back-face of ray-cube to a texture and returns it"""

    # render to FBO

    glBindFramebuffer(GL_FRAMEBUFFER, self.fboHandle)

    # set active texture

    glActiveTexture(GL_TEXTURE0)

    # bind to FBO texture

    glBindTexture(GL_TEXTURE_2D, self.texHandle)

    # render cube with face culling enabled

    self.renderCube(pMatrix, mvMatrix, self.program, True)

    # unbind texture
```

```
glBindTexture(GL_TEXTURE_2D, 0)

glBindFramebuffer(GL_FRAMEBUFFER, 0)

glBindRenderbuffer(GL_RENDERBUFFER, 0)

# return texture ID

return self.texHandle

def renderFrontFace(self, pMatrix, mvMatrix, program):

    """render front-face of ray-cube"""

    # no face culling

    self.renderCube(pMatrix, mvMatrix, program, False)

def renderCube(self, pMatrix, mvMatrix, program, cullFace):

    """render cube using face culling if flag set"""

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

    # set shader program

    glUseProgram(program)

    # set projection matrix

    glUniformMatrix4fv(glGetUniformLocation(program,
b'uPMatrix'),

                        1, GL_FALSE, pMatrix)

    # set modelview matrix
```

```
glUniformMatrix4fv(glGetUniformLocation(program,  
b'uMVMMatrix'),
```

```
1, GL_FALSE, mvMatrix)
```

```
# enable face culling
```

```
glDisable(GL_CULL_FACE)
```

```
if cullFace:
```

```
    glFrontFace(GL_CCW)
```

```
    glCullFace(GL_FRONT)
```

```
    glEnable(GL_CULL_FACE)
```

```
# bind VAO
```

```
glBindVertexArray(self.vao)
```

```
# animated slice
```

```
glDrawElements(GL_TRIANGLES, self.nIndices,  
GL_UNSIGNED_SHORT, None)
```

```
# unbind VAO
```

```
glBindVertexArray(0)
```

```
# reset cull face
```

```
if cullFace:
```

```
    # disable face culling
```

```
    glDisable(GL_CULL_FACE)
```

```
def reshape(self, width, height):

    self.width = width

    self.height = height

    self.aspect = width/float(height)

    # re-create FBO

    self.clearFBO()

    self.initFBO()

def initFBO(self):

    # create frame buffer object

    self.fboHandle = glGenFramebuffers(1)

    # create texture

    self.texHandle = glGenTextures(1)

    # create depth buffer

    self.depthHandle = glGenRenderbuffers(1)

    # bind

    glBindFramebuffer(GL_FRAMEBUFFER, self.fboHandle)

    glActiveTexture(GL_TEXTURE0)

    glBindTexture(GL_TEXTURE_2D, self.texHandle)

    # set parameters to draw the image at different sizes
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
GL_LINEAR)
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,  
GL_LINEAR)
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,  
GL_CLAMP_TO_EDGE)
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,  
GL_CLAMP_TO_EDGE)
```

```
# set up texture
```

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, self.width,  
self.height,
```

```
0, GL_RGBA, GL_UNSIGNED_BYTE, None)
```

```
# bind texture to FBO
```

```
glFramebufferTexture2D(GL_FRAMEBUFFER,  
GL_COLOR_ATTACHMENT0,
```

```
GL_TEXTURE_2D, self.texHandle, 0)
```

```
# bind
```

```
glBindRenderbuffer(GL_RENDERBUFFER, self.depthHandle)
```

```
glRenderbufferStorage(GL_RENDERBUFFER,  
GL_DEPTH_COMPONENT24,
```

```
self.width, self.height)
```

```
# bind depth buffer to FBO
```

```
        glFramebufferRenderbuffer(GL_FRAMEBUFFER,
        GL_DEPTH_ATTACHMENT,

        GL_RENDERBUFFER, self.depthHandle)

    # check status

    status = glCheckFramebufferStatus(GL_FRAMEBUFFER)

    if status == GL_FRAMEBUFFER_COMPLETE:

        pass

        # print "fbo %d complete" % self.fboHandle

    elif status == GL_FRAMEBUFFER_UNSUPPORTED:

        print("fbo %d unsupported" % self.fboHandle)

    else:

        print("fbo %d Error" % self.fboHandle)

    glBindTexture(GL_TEXTURE_2D, 0)

    glBindFramebuffer(GL_FRAMEBUFFER, 0)

    glBindRenderbuffer(GL_RENDERBUFFER, 0)

    return

def clearFBO(self):

    """clears old FBO"""

    # delete FBO
```

```
if glIsFramebuffer(self.fboHandle):

    glDeleteFramebuffers(1, int(self.fboHandle))

# delete texture

if glIsTexture(self.texHandle):

    glDeleteTextures(int(self.texHandle))

def close(self):

    """call this to free up OpenGL resources"""

    glBindTexture(GL_TEXTURE_2D, 0)

    glBindFramebuffer(GL_FRAMEBUFFER, 0)

    glBindRenderbuffer(GL_RENDERBUFFER, 0)

    # delete FBO

    if glIsFramebuffer(self.fboHandle):

        glDeleteFramebuffers(1, int(self.fboHandle))

    # delete texture

    if glIsTexture(self.texHandle):

        glDeleteTextures(int(self.texHandle))

    # delete render buffer

    """

    if glIsRenderbuffer(self.depthHandle):
```

```
glDeleteRenderbuffers(1, int(self.depthHandle))

"""

# delete buffers

"""

glDeleteBuffers(1, self._vertexBuffer)

glDeleteBuffers(1, &_amp;_indexBuffer)

glDeleteBuffers(1, &_amp;_colorBuffer)

"""
```

The Complete Volume Ray Casting Code

Here's the full *raycast.py* code listing.

```
import OpenGL

from OpenGL.GL import *

from OpenGL.GL.shaders import *

import numpy as np

import math, sys

import raycube, glutils, volreader

strVS = """

# version 330 core
```

```
layout(location = 1) in vec3 cubePos;

layout(location = 2) in vec3 cubeCol;

uniform mat4 uMVMMatrix;

uniform mat4 uPMatrix;

out vec4 vColor;

void main()

{

    // set position

    gl_Position = uPMatrix * uMVMMatrix * vec4(cubePos.xyz, 1.0);

    // set color

    vColor = vec4(cubeCol.rgb, 1.0);

}

"""

strFS = """

# version 330 core

in vec4 vColor;

uniform sampler2D texBackFaces;

uniform sampler3D texVolume;

uniform vec2 uWinDims;
```

```
out vec4 fragColor;

void main()

{

    // start of ray

    vec3 start = vColor.rgb;

    // calculate texture coords at fragment,

    // which is a fraction of window coords

    vec2 texc = gl_FragCoord.xy/uWinDims.xy;

    // get end of ray by looking up back-face color

    vec3 end = texture(texBackFaces, texc).rgb;

    // calculate ray direction

    vec3 dir = end - start;

    // normalized ray direction

    vec3 norm_dir = normalize(dir);

    // the length from front to back is calculated and

    // used to terminate the ray

    float len = length(dir.xyz);

    // ray step size

    float stepSize = 0.01;
```

```
// X-ray projection

vec4 dst = vec4(0.0);

// step through the ray

for(float t = 0.0; t < len; t += stepSize) {

    // set position to endpoint of ray

    vec3 samplePos = start + t*norm_dir;

    // get texture value at position

    float val = texture(texVolume, samplePos).r;

    vec4 src = vec4(val);

    // set opacity

    src.a *= 0.1;

    src.rgb *= src.a;

    // blend with previous value

    dst = (1.0 - dst.a)*src + dst;

    // exit loop when alpha exceeds threshold

    if(dst.a >= 0.95)

        break;

}
```



```
// set fragment color
```

```
    fragColor = dst;

}

"""

class Camera:

    """helper class for viewing"""

    def __init__(self):

        self.r = 1.5

        self.theta = 0

        self.center = [0.5, 0.5, 0.5]

        self.eye = [0.5 + self.r, 0.5, 0.5]

        self.up = [0.0, 0.0, 1.0]

    def rotate(self, clockWise):

        """rotate eye by one step"""

        if clockWise:

            self.theta = (self.theta + 5) % 360

        else:

            self.theta = (self.theta - 5) % 360

        # recalculate eye

        self.eye = [0.5 + self.r*math.cos(math.radians(self.theta)),
```

```
0.5 + self.r*math.sin(math.radians(self.theta)),  
  
0.5]
```

```
class RayCastRender:
```

```
    """class that does Ray Casting"""
```

```
    def __init__(self, width, height, volume):
```

```
        """RayCastRender constr"""
```

```
        # create RayCube object
```

```
        self.raycube = raycube.RayCube(width, height)
```

```
        # set dimensions
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.aspect = width/float(height)
```

```
        # create shader
```

```
        self.program = glutils.loadShaders(strVS, strFS)
```

```
        # texture
```

```
        self.texVolume, self.Nx, self.Ny, self.Nz = volume
```

```
        # initialize camera
```

```
        self.camera = Camera()
```

```
    def draw(self):
```

```
# build projection matrix

pMatrix = glutils.perspective(45.0, self.aspect, 0.1, 100.0)

# modelview matrix

mvMatrix = glutils.lookAt(self.camera.eye, self.camera.center,

                           self.camera.up)

# render

# generate ray-cube back-face texture

texture = self.raycube.renderBackFace(pMatrix, mvMatrix)

# set shader program

glUseProgram(self.program)

# set window dimensions

glUniform2f(glGetUniformLocation(self.program, b"uWinDims"),

            float(self.width), float(self.height))

# texture unit 0, which represents back-faces of cube

glActiveTexture(GL_TEXTURE0)

glBindTexture(GL_TEXTURE_2D, texture)

glUniform1i(glGetUniformLocation(self.program,

b"texBackFaces"), 0)

# texture unit 1: 3D volume texture
```

```
glActiveTexture(GL_TEXTURE1)

glBindTexture(GL_TEXTURE_3D, self.texVolume)

glUniform1i(glGetUniformLocation(self.program, b"texVolume"),
1)

# draw front-face of cubes

self.raycube.renderFrontFace(pMatrix, mvMatrix, self.program)

#self.render(pMatrix, mvMatrix)

def keyPressed(self, key):

    if key == 'l':

        self.camera.rotate(True)

    elif key == 'r':

        self.camera.rotate(False)

def reshape(self, width, height):

    self.width = width

    self.height = height

    self.aspect = width/float(height)

    self.raycube.reshape(width, height)

def close(self):

    self.raycube.close()
```

The Complete 2D Slicing Code

Here is the full 2D slicing code listing.

```
import OpenGL

from OpenGL.GL import *

from OpenGL.GL.shaders import *

import numpy, math, sys

import volreader, glutils

strVS = ""

# version 330 core

in vec3 aVert;

uniform mat4 uMVMatrix;

uniform mat4 uPMatrix;

uniform float uSliceFrac;

uniform int uSliceMode;

out vec3 texcoord;

void main() {

    // x slice

    if (uSliceMode == 0) {

        texcoord = vec3(uSliceFrac, aVert.x, 1.0-aVert.y);
```

```
}

// y slice

else if (uSliceMode == 1) {

    texcoord = vec3(aVert.x, uSliceFrac, 1.0-aVert.y);

}

// z slice

else {

    texcoord = vec3(aVert.x, 1.0-aVert.y, uSliceFrac);

}

// calculate transformed vertex

gl_Position = uPMatrix * uMVMatrix * vec4(aVert, 1.0);

}

"""

strFS = """

# version 330 core

in vec3 texcoord;

uniform sampler3D tex;

out vec4 fragColor;

void main() {
```

```
// look up color in texture

vec4 col = texture(tex, texcoord);

fragColor = col.rrra;

}

"""

class SliceRender:

    # slice modes

    XSLICE, YSLICE, ZSLICE = 0, 1, 2

    def __init__(self, width, height, volume):

        """SliceRender constructor"""

        self.width = width

        self.height = height

        self.aspect = width/float(height)

        # slice mode

        self.mode = SliceRender.ZSLICE

        # create shader

        self.program = glutils.loadShaders(strVS, strFS)

        glUseProgram(self.program)
```

```
self.pMatrixUniform = glGetUniformLocation(self.program,
b'uPMatrix')

self.mvMatrixUniform = glGetUniformLocation(self.program,
b"uMVMatrix")

# attributes

self.vertIndex = glGetAttribLocation(self.program, b"aVert")

# set up vertex array object (VAO)

self.vao = glGenVertexArrays(1)

glBindVertexArray(self.vao)

# define quad vertices

vertexData = numpy.array([0.0, 1.0, 0.0,

                           0.0, 0.0, 0.0,

                           1.0, 1.0, 0.0,

                           1.0, 0.0, 0.0], numpy.float32)

# vertex buffer

self.vertexBuffer = glGenBuffers(1)

glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

glBufferData(GL_ARRAY_BUFFER, 4*len(vertexData), vertexData,

             GL_STATIC_DRAW)

# enable arrays
```

```
glEnableVertexAttribArray(self.vertIndex)

# set buffers

glBindBuffer(GL_ARRAY_BUFFER, self.vertexBuffer)

glVertexAttribPointer(self.vertIndex, 3, GL_FLOAT, GL_FALSE, 0,
None)

# unbind VAO

glBindVertexArray(0)

# load texture

self.texture, self.Nx, self.Ny, self.Nz = volume

# current slice index

self.currSliceIndex = int(self.Nz/2);

self.currSliceMax = self.Nz;

def reshape(self, width, height):

    self.width = width

    self.height = height

    self.aspect = width/float(height)

def draw(self):

    # clear buffers

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
# build projection matrix

pMatrix = glutils.ortho(-0.6, 0.6, -0.6, 0.6, 0.1, 100.0)

# modelview matrix

mvMatrix = numpy.array([1.0, 0.0, 0.0, 0.0,

                        0.0, 1.0, 0.0, 0.0,

                        0.0, 0.0, 1.0, 0.0,

                        -0.5, -0.5, -1.0, 1.0], numpy.float32)

# use shader

glUseProgram(self.program)

# set projection matrix

glUniformMatrix4fv(self.pMatrixUniform, 1, GL_FALSE, pMatrix)

# set modelview matrix

glUniformMatrix4fv(self.mvMatrixUniform, 1, GL_FALSE,
mvMatrix)

# set current slice fraction

glUniform1f(glGetUniformLocation(self.program, b"uSliceFrac"),

            float(self.currSliceIndex)/float(self.currSliceMax))

# set current slice mode

glUniform1i(glGetUniformLocation(self.program, b"uSliceMode"),
```

```
        self.mode)

    # enable texture

    glActiveTexture(GL_TEXTURE0)

    glBindTexture(GL_TEXTURE_3D, self.texture)

    glUniform1i(glGetUniformLocation(self.program, b"tex"), 0)

    # bind VAO

    glBindVertexArray(self.vao)

    # draw

    glDrawArrays(GL_TRIANGLE_STRIP, 0, 4)

    # unbind VAO

    glBindVertexArray(0)

def keyPressed(self, key):

    """keypress handler"""

    if key == 'x':

        self.mode = SliceRender.XSLICE

        # reset slice index

        self.currSliceIndex = int(self.Nx/2)

        self.currSliceMax = self.Nx

    elif key == 'y':
```

```
self.mode = SliceRender.YSLICE

# reset slice index

self.currSliceIndex = int(self.Ny/2)

self.currSliceMax = self.Ny

elif key == 'z':

    self.mode = SliceRender.ZSLICE

    # reset slice index

    self.currSliceIndex = int(self.Nz/2)

    self.currSliceMax = self.Nz

elif key == 'l':

    self.currSliceIndex = (self.currSliceIndex + 1) %
self.currSliceMax

elif key == 'r':

    self.currSliceIndex = (self.currSliceIndex - 1) %
self.currSliceMax

def close(self):

    pass
```

The Complete Main File Code

Here is the full code listing for the main file.

```
import sys, argparse, os

from slicerender import *

from raycast import *

import glfw

class RenderWin:

    """GLFW Rendering window class"""

    def __init__(self, imageDir):

        # save current working directory

        cwd = os.getcwd()

        # initialize glfw; this changes cwd

        glfw.glfwInit()

        # restore cwd

        os.chdir(cwd)

        # version hints

        glfw.glfwWindowHint(glfw.GLFW_CONTEXT_VERSION_MAJOR, 3)

        glfw.glfwWindowHint(glfw.GLFW_CONTEXT_VERSION_MINOR, 3)

        glfw.glfwWindowHint(glfw.GLFW_OPENGL_FORWARD_COMPAT,
GL_TRUE)

        glfw.glfwWindowHint(glfw.GLFW_OPENGL_PROFILE,
```

```
glfw.Glfw._OPENGGL_CORE_PROFILE)

# make a window

self.width, self.height = 512, 512

self.aspect = self.width/float(self.height)

self.win = glfw.glfwCreateWindow(self.width, self.height,
b"volrender")

# make context current

glfw.glfwMakeContextCurrent(self.win)

# initialize GL

glViewport(0, 0, self.width, self.height)

glEnable(GL_DEPTH_TEST)

glClearColor(0.0, 0.0, 0.0, 0.0)

# set window callbacks

glfw.glfwSetMouseButtonCallback(self.win, self.onMouseButton)

glfw.glfwSetKeyCallback(self.win, self.onKeyboard)

glfw.glfwSetWindowSizeCallback(self.win, self.onSize)

# load volume data

self.volume = volreader.loadVolume(imageDir)

# create renderer
```

```
self.renderer = RayCastRender(self.width, self.height, self.volume)

# exit flag

self.exitNow = False

def onMouseButton(self, win, button, action, mods):

    # print 'mouse button: ', win, button, action, mods

    pass

def onKeyboard(self, win, key, scancode, action, mods):

    # print 'keyboard: ', win, key, scancode, action, mods

    # ESC to quit

    if key is glfw.GLFW_KEY_ESCAPE:

        self.renderer.close()

        self.exitNow = True

    else:

        if action is glfw.GLFW_PRESS or action is glfw.GLFW_REPEAT:

            if key == glfw.GLFW_KEY_V:

                # toggle render mode

                if isinstance(self.renderer, RayCastRender):

                    self.renderer = SliceRender(self.width, self.height,

                                                self.volume)
```

else:

```
self.renderer = RayCastRender(self.width, self.height,  
  
                                self.volume)
```

call reshape on renderer

```
self.renderer.reshape(self.width, self.height)
```

else:

send keypress to renderer

```
keyDict = {glfw.GLFW_KEY_X: 'x', glfw.GLFW_KEY_Y: 'y',  
  
            glfw.GLFW_KEY_Z: 'z', glfw.GLFW_KEY_LEFT: 'l',  
  
            glfw.GLFW_KEY_RIGHT: 'r'}
```

try:

```
self.renderer.keyPressed(keyDict[key])
```

except:

pass

def onSize(self, win, width, height):

print 'onsize: ', win, width, height

```
self.width = width
```

```
self.height = height
```

```
self.aspect = width/float(height)
```

```
glViewport(0, 0, self.width, self.height)

self.renderer.reshape(width, height)

def run(self):

    # start loop

    while not glfw.glfwWindowShouldClose(self.win) and not
self.exitNow:

        # render

        self.renderer.draw()

        # swap buffers

        glfw.glfwSwapBuffers(self.win)

        # wait for events

        glfw.glfwWaitEvents()

    # end

    glfw.glfwTerminate()

# main() function

def main():

    print('starting volrender...')

    # create parser

    parser = argparse.ArgumentParser(description="Volume
Rendering...")
```

```
# add expected arguments

parser.add_argument('--dir', dest='imageDir', required=True)

# parse args

args = parser.parse_args()

# create render window

rwin = RenderWin(args.imageDir)

rwin.run()

# call main

if __name__ == '__main__':

    main()
```

1 J. Kruger and R. Westermann, “Acceleration Techniques for GPU-based Volume Rendering,” IEEE Visualization, 2003.

2 <https://graphics.stanford.edu/data/voldata/>